

1. Basic Coding — 1 to 20		
#	Question	Main Concept
1	Reverse a String	String
2	Check String Palindrome	String
3	Check Number Palindrome	Number
4	Reverse an Integer	Number
5	Check Prime Number	Math
6	Print Prime Numbers in Range	Math
7	Factorial of Number	Loop
8	Fibonacci Series	Loop
9	Nth Fibonacci Number	Recursion/Loop
10	Armstrong Number	Math
11	Perfect Number	Math
12	GCD of Two Numbers	Math
13	LCM of Two Numbers	Math
14	Count Digits	Number
15	Sum of Digits	Number
16	Reverse Words in String	String
17	Count Vowels and Consonants	String
18	Character Frequency	Hashing
19	Duplicate Characters	Hashing
20	Remove Duplicate Characters	Hashing

1. Reverse a String

```
</> C++  
  
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    string s = "hello";  
  
    reverse(s.begin(), s.end());  
  
    cout << s;  
}
```

Output:

olleh

2. String Palindrome

```
</> C++  
  
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    string s = "madam";  
  
    string rev = s;  
    reverse(rev.begin(), rev.end());  
  
    if(s == rev)  
        cout << "Palindrome";  
    else  
        cout << "Not Palindrome";  
}
```

3. Number Palindrome

```
</> C++

#include <bits/stdc++.h>
using namespace std;

int main() {
    int n = 121;
    int original = n;
    int rev = 0;

    while(n > 0) {
        int digit = n % 10;
        rev = rev * 10 + digit;
        n /= 10;
    }

    if(original == rev)
        cout << "Palindrome";
    else
        cout << "Not Palindrome";
}
```

4. Reverse an Integer

```
</> C++

int n = 12345;
int rev = 0;

while(n > 0) {
    rev = rev * 10 + n % 10;
    n /= 10;
}

cout << rev;
```

5. Prime Number

```
</> C++  
  
bool isPrime(int n) {  
    if(n < 2)  
        return false;  
  
    for(int i = 2; i * i <= n; i++) {  
        if(n % i == 0)  
            return false;  
    }  
  
    return true;  
}
```

6. Print Prime Numbers in Range

```
</> C++  
  
for(int n = 2; n <= 50; n++) {  
  
    bool prime = true;  
  
    for(int i = 2; i * i <= n; i++) {  
        if(n % i == 0) {  
            prime = false;  
            break;  
        }  
    }  
  
    if(prime)  
        cout << n << " ";  
  
}
```

7. Factorial

```
</> C++  
  
int n = 5;  
long long fact = 1;  
  
for(int i = 1; i <= n; i++)  
    fact *= i;  
  
cout << fact;
```

Output:

120

8. Fibonacci Series

```
</> C++  
  
int n = 10;  
  
int a = 0, b = 1;  
  
for(int i = 0; i < n; i++) {  
    cout << a << " ";  
  
    int c = a + b;  
    a = b;  
    b = c;  
}
```

Output:

0 1 1 2 3 5 8 13 21 34

9. Nth Fibonacci

```
</> C++  
  
int fib(int n) {  
  
    if(n <= 1)  
        return n;  
  
    int a = 0, b = 1;  
  
    for(int i = 2; i <= n; i++) {  
        int c = a + b;  
        a = b;  
        b = c;  
    }  
  
    return b;  
}
```

10. Armstrong Number

```
</> C++  
  
int n = 153;  
int original = n;  
int sum = 0;  
  
while(n > 0) {  
    int d = n % 10;  
    sum += d * d * d;  
    n /= 10;  
}  
  
if(sum == original)  
    cout << "Armstrong";  
else  
    cout << "Not Armstrong";
```

11. Perfect Number

```
</> C++  
  
int n = 28;  
int sum = 0;  
  
for(int i = 1; i < n; i++) {  
    if(n % i == 0)  
        sum += i;  
}  
  
if(sum == n)  
    cout << "Perfect";  
else  
    cout << "Not Perfect";
```

12. GCD

```
</> C++  
  
int a = 12, b = 18;  
  
while(b != 0) {  
    int temp = b;  
    b = a % b;  
    a = temp;  
}  
  
cout << a;
```

Output:

6

Or STL:

```
</> C++  
  
cout << gcd(12, 18);
```

13. LCM

```
</> C++  
  
int a = 12, b = 18;  
  
int g = gcd(a, b);  
  
int lcm = (a / g) * b;  
  
cout << lcm;
```

14. Count Digits

```
</> C++  
  
int n = 12345;  
int count = 0;  
  
while(n > 0) {  
    count++;  
    n /= 10;  
}  
  
cout << count;
```

15. Sum of Digits

</> C++

```
int n = 12345;  
int sum = 0;  
  
while(n > 0) {  
    sum += n % 10;  
    n /= 10;  
}  
  
cout << sum;
```

16. Reverse Words

```
</> C++  
  
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
  
    string s = "I love C++";  
  
    stringstream ss(s);  
    string word;  
  
    vector<string> v;  
  
    while(ss >> word)  
        v.push_back(word);  
  
    reverse(v.begin(), v.end());  
  
    for(auto x : v)  
        cout << x << " ";  
}
```

17. Count Vowels & Consonants

```
</> C++

string s = "hello";
int vowel = 0, consonant = 0;

for(char c : s) {

    if(isalpha(c)) {

        c = tolower(c);

        if(c=='a' || c=='e' || c=='i' || c=='o' || c=='u')
            vowel++;
        else
            consonant++;
    }
}

cout << vowel << " " << consonant;
```

18. Character Frequency

```
</> C++

string s = "hello";

map<char,int> freq;

for(char c : s)
    freq[c]++;

for(auto p : freq)
    cout << p.first << " : " << p.second << endl;
```

19. Duplicate Characters

```
</> C++  
  
string s = "programming";  
  
map<char,int> freq;  
  
for(char c : s)  
    freq[c]++;  
  
for(auto p : freq) {  
    if(p.second > 1)  
        cout << p.first << " ";  
}
```

20. Remove Duplicate Characters

```
</> C++  
  
string s = "programming";  
  
unordered_set<char> st;  
  
for(char c : s)  
    st.insert(c);  
  
for(char c : st)  
    cout << c;
```

Order preserve karna ho:

Order preserve karna ho:

```
</> C++

string ans;
unordered_set<char> seen;

for(char c : s) {
    if(!seen.count(c)) {
        ans += c;
        seen.insert(c);
    }
}
```

Array Questions — 21 to 40

#	Question	Concept
21	Find Largest Element	Array
22	Find Smallest Element	Array
23	Second Largest	Array
24	Second Smallest	Array
25	Sum of Array	Array
26	Reverse Array	Two Pointer
27	Check Sorted Array	Array
28	Remove Duplicates	Array
29	Find Duplicate Element	Hashing
30	Find Missing Number	Math/XOR

30	Find Missing Number	Math/XOR
31	Frequency of Elements	Map
32	Common Elements	Set
33	Merge Two Arrays	Array
34	Union of Two Arrays	Set
35	Intersection of Two Arrays	Set
36	Move Zeros to End	Two Pointer
37	Maximum-Minimum Difference	Array
38	Pair With Given Sum	Two Sum
39	Maximum Subarray Sum	Kadane
40	Rotate Array	Array



21. Largest Element

```
</> C++  
  
vector<int> a = {10, 5, 30, 20};  
  
int largest = a[0];  
  
for(int x : a)  
    largest = max(largest, x);  
  
cout << largest;
```

22. Smallest Element

```
</> C++  
  
int smallest = a[0];  
  
for(int x : a)  
    smallest = min(smallest, x);  
  
cout << smallest;
```

23. Second Largest

```
</> C++  
  
vector<int> a = {10, 5, 30, 20};  
  
int largest = INT_MIN;  
int second = INT_MIN;  
  
for(int x : a) {  
  
    if(x > largest) {  
        second = largest;  
        largest = x;  
    }  
    else if(x > second && x != largest) {  
        second = x;  
    }  
}  
  
cout << second;
```

24. Second Smallest

```
</> C++

int smallest = INT_MAX;
int second = INT_MAX;

for(int x : a) {

    if(x < smallest) {
        second = smallest;
        smallest = x;
    }
    else if(x < second && x != smallest) {
        second = x;
    }
}

cout << second;
```

25. Sum of Array

```
</> C++

int sum = 0;

for(int x : a)
    sum += x;

cout << sum;
```

26. Reverse Array

```
</> C++  
  
int i = 0;  
int j = a.size() - 1;  
  
while(i < j) {  
    swap(a[i], a[j]);  
    i++;  
    j--;  
}
```

27. Check Sorted Array

```
</> C++  
  
bool sorted = true;  
  
for(int i = 1; i < a.size(); i++) {  
  
    if(a[i] < a[i-1]) {  
        sorted = false;  
        break;  
    }  
}  
  
cout << (sorted ? "Sorted" : "Not Sorted");
```

28. Remove Duplicates

```
</> C++  
  
sort(a.begin(), a.end());  
  
a.erase(unique(a.begin(), a.end()), a.end());  
  
for(int x : a)  
    cout << x << " ";
```

29. Find Duplicate Element

```
</> C++  
  
vector<int> a = {1,2,3,4,2};  
  
unordered_set<int> st;  
  
for(int x : a) {  
  
    if(st.count(x)) {  
        cout << "Duplicate: " << x;  
        break;  
    }  
  
    st.insert(x);  
}
```

30. Missing Number

Array contains `1...n`, one number missing.

```
</> C++  
  
vector<int> a = {1,2,3,5};  
  
int n = 5;  
  
int expected = n * (n + 1) / 2;  
  
int actual = 0;  
  
for(int x : a)  
    actual += x;  
  
cout << expected - actual;
```

31. Frequency of Array Elements

```
</> C++  
  
map<int,int> freq;  
  
for(int x : a)  
    freq[x]++;  
  
for(auto p : freq)  
    cout << p.first << " : " << p.second << endl;
```

32. Common Elements

```
</> C++  
  
vector<int> a = {1,2,3,4};  
vector<int> b = {3,4,5,6};  
  
set<int> s(a.begin(), a.end());  
  
for(int x : b) {  
  
    if(s.count(x))  
        cout << x << " ";  
  
}
```

33. Merge Two Arrays

```
</> C++  
  
vector<int> a = {1,2,3};  
vector<int> b = {4,5,6};  
  
vector<int> c = a;  
  
c.insert(c.end(), b.begin(), b.end());  
  
for(int x : c)  
    cout << x << " ";
```

34. Union of Two Arrays

```
</> C++  
  
set<int> s;  
  
for(int x : a)  
    s.insert(x);  
  
for(int x : b)  
    s.insert(x);  
  
for(int x : s)  
    cout << x << " ";
```

35. Intersection

```
</> C++  
  
set<int> s(a.begin(), a.end());  
  
for(int x : b) {  
  
    if(s.count(x))  
        cout << x << " ";  
  
}
```

36. Move Zeros to End

```
</> C++  
  
vector<int> a = {0,1,0,3,12};  
  
int j = 0;  
  
for(int i = 0; i < a.size(); i++) {  
  
    if(a[i] != 0) {  
        swap(a[i], a[j]);  
        j++;  
    }  
}  
  
for(int x : a)  
    cout << x << " ";
```

Output:

```
1 3 12 0 0
```

37. Maximum-Minimum Difference

```
</> C++  
  
int mn = *min_element(a.begin(), a.end());  
int mx = *max_element(a.begin(), a.end());  
  
cout << mx - mn;
```

38. Pair With Given Sum — Two Sum

```
</> C++  
  
vector<int> a = {2,7,11,15};  
int target = 9;  
  
unordered_set<int> st;  
  
for(int x : a) {  
  
    int need = target - x;  
  
    if(st.count(need)) {  
        cout << need << " " << x;  
        break;  
    }  
  
    st.insert(x);  
}
```

Very commonly asked.

39. Maximum Subarray Sum — Kadane

```
</> C++  
  
vector<int> a = {-2,1,-3,4,-1,2,1,-5,4};  
  
int current = a[0];  
int maximum = a[0];  
  
for(int i = 1; i < a.size(); i++) {  
  
    current = max(a[i], current + a[i]);  
  
    maximum = max(maximum, current);  
}  
  
cout << maximum;
```

Output:

6

40. Rotate Array

Right rotate by `k` .

```
</> C++  
  
vector<int> a = {1,2,3,4,5};  
int k = 2;  
  
k %= a.size();  
  
reverse(a.begin(), a.end());  
  
reverse(a.begin(), a.begin()+k);  
  
reverse(a.begin()+k, a.end());  
  
for(int x : a)  
    cout << x << " ";
```

Output:

```
4 5 1 2 3
```

🔥 Searching + Sorting — 41 to 50

#	Question	Concept
41	Linear Search	Searching
42	Binary Search	Searching
43	First Occurrence	Binary Search
44	Last Occurrence	Binary Search
45	Bubble Sort	Sorting
46	Selection Sort	Sorting
47	Insertion Sort	Sorting
48	Merge Sort	Divide & Conquer
49	Quick Sort	Divide & Conquer
50	Sort 0,1,2	Dutch National Flag



41. Linear Search

```

</> C++

int target = 30;

for(int i = 0; i < a.size(); i++) {

    if(a[i] == target) {
        cout << "Found at index " << i;
        break;
    }
}

```

42. Binary Search

Array sorted hona chahiye.

```
</> C++  
  
vector<int> a = {10,20,30,40,50};  
  
int target = 40;  
  
int low = 0;  
int high = a.size()-1;  
  
while(low <= high) {  
  
    int mid = low + (high-low)/2;  
  
    if(a[mid] == target) {  
        cout << mid;  
        break;  
    }  
  
    else if(a[mid] < target)  
        low = mid + 1;  
}
```

43. First Occurrence

```
</> C++

int low = 0, high = a.size()-1;
int ans = -1;

while(low <= high) {

    int mid = low + (high-low)/2;

    if(a[mid] == target) {
        ans = mid;
        high = mid - 1;
    }
    else if(a[mid] < target)
        low = mid + 1;
    else
        high = mid - 1;
}

cout << ans;
```

44. Last Occurrence

```
</> C++

int low = 0, high = a.size()-1;
int ans = -1;

while(low <= high) {

    int mid = low + (high-low)/2;

    if(a[mid] == target) {
        ans = mid;
        low = mid + 1;
    }
    else if(a[mid] < target)
        low = mid + 1;
    else
        high = mid - 1;
}

cout << ans;
```

45. Bubble Sort

```
</> C++

for(int i = 0; i < a.size(); i++) {

    for(int j = 0; j < a.size()-i-1; j++) {

        if(a[j] > a[j+1])
            swap(a[j], a[j+1]);
    }
}
```

46. Selection Sort

```
</> C++  
  
for(int i = 0; i < a.size(); i++) {  
  
    int minIndex = i;  
  
    for(int j = i+1; j < a.size(); j++) {  
  
        if(a[j] < a[minIndex])  
            minIndex = j;  
    }  
  
    swap(a[i], a[minIndex]);  
}
```

47. Insertion Sort

```
</> C++  
  
for(int i = 1; i < a.size(); i++) {  
  
    int key = a[i];  
    int j = i-1;  
  
    while(j >= 0 && a[j] > key) {  
        a[j+1] = a[j];  
        j--;  
    }  
  
    a[j+1] = key;  
}
```

48. Merge Sort

```
</> C++  
  
void mergeSort(vector<int>& a, int l, int r) {  
  
    if(l >= r)  
        return;  
  
    int mid = l + (r-l)/2;  
  
    mergeSort(a,l,mid);  
    mergeSort(a,mid+1,r);  
  
    vector<int> temp;  
  
    int i=l, j=mid+1;  
  
    while(i<=mid && j<=r) {  
  
        if(a[i] < a[j])  
            temp.push_back(a[i++]);  
        else  
            temp.push_back(a[j++]);  
    }  
}
```



Coding_part_1

```
        temp.push_back(a[i++]);
    else
        temp.push_back(a[j++]);
    }

    while(i<=mid)
        temp.push_back(a[i++]);

    while(j<=r)
        temp.push_back(a[j++]);

    for(int k=0; k<temp.size(); k++)
        a[l+k] = temp[k];
}
```

Call:

`</> C++`

```
mergeSort(a,0,a.size()-1);
```

49. Quick Sort

</> C++

```
void quickSort(vector<int>& a, int low, int high) {  
  
    if(low >= high)  
        return;  
  
    int pivot = a[high];  
  
    int i = low;  
  
    for(int j=low; j<high; j++) {  
  
        if(a[j] < pivot) {  
            swap(a[i],a[j]);  
            i++;  
        }  
    }  
  
    swap(a[i],a[high]);  
  
    quickSort(a,low,i-1);  
    quickSort(a,i+1,high);  
}
```



50. Sort 0, 1, 2

```
</> C++  
  
vector<int> a = {0,2,1,2,0,1};  
  
int low = 0;  
int mid = 0;  
int high = a.size()-1;  
  
while(mid <= high) {  
  
    if(a[mid] == 0)  
        swap(a[low++],a[mid++]);  
  
    else if(a[mid] == 1)  
        mid++;  
  
    else  
        swap(a[mid],a[high--]);  
}
```

Top 15 Hashing Questions

#	Question	Main STL
1	Frequency of Array Elements	map
2	Frequency of Characters	map
3	Find Duplicate Elements	set
4	Remove Duplicates from Array	set
5	Find Missing Number	set
6	Two Sum	unordered_map ★
7	First Non-Repeating Character	map
8	First Repeating Element	set
9	Common Elements of Two Arrays	set
8	Find the Element which is Missing from the Array	
9	Count the Number of Elements which are Greater than or Equal to X	
10	Union of Two Arrays	
11	Check if Two Arrays are Disjoint	

9	Common Elements of Two Arrays	set
10	Union of Two Arrays	set
11	Check Anagram	map
12	Group Anagrams	map
13	Longest Consecutive Sequence	unordered_set ★
14	Count Subarrays with Given Sum	unordered_map ★
15	Longest Subarray with Sum K	unordered_map ★

1. Frequency of Array Elements ★

Input

1 2 2 3 3 3

Code

```
</> C++  
  
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
  
    vector<int> a = {1,2,2,3,3,3};  
  
    map<int,int> freq;  
  
    for(int x : a) {  
        freq[x]++;  
    }
```



Coding_part_1

⌕ C++

```
for(auto p : freq) {  
    cout << p.first << " -> " << p.second << endl;  
}  
}
```

Output:

```
1 -> 1  
2 -> 2  
3 -> 3
```

Important

⌕ C++

```
freq[x]++;
```

Matlab:

```
x mila -> uski frequency +1
```



2. Frequency of Characters

```
</> C++  
  
string s = "programming";  
  
map<char,int> freq;  
  
for(char c : s) {  
    freq[c]++;  
}  
  
for(auto p : freq) {  
    cout << p.first << " -> " << p.second << endl;  
}
```

Useful for:

- duplicate characters
- anagram
- frequency problems

3. Find Duplicate Elements

```
</> C++  
  
vector<int> a = {1,2,3,2,4,3,5};  
  
unordered_set<int> st;  
  
for(int x : a) {  
  
    if(st.count(x)) {  
        cout << x << " ";  
    }  
  
    st.insert(x);  
}
```

Output:

```
2 3
```

4. Remove Duplicates from Array

```
</> C++  
  
vector<int> a = {1,2,2,3,3,4};  
  
set<int> st(a.begin(), a.end());  
  
for(int x : st) {  
    cout << x << " ";  
}
```

Output:

```
1 2 3 4
```

If order doesn't matter → `set` .

If original order preserve karna hai:

If original order preserve karna hai:

```
</> C++  
  
vector<int> ans;  
unordered_set<int> seen;  
  
for(int x : a) {  
  
    if(!seen.count(x)) {  
        ans.push_back(x);  
        seen.insert(x);  
    }  
}  
  
for(int x : ans)  
    cout << x << " ";
```

5. Find Missing Number

Given:

1 2 3 5

Missing:

4

Using Set

```
</> C++  
  
vector<int> a = {1,2,3,5};  
  
unordered_set<int> st(a.begin(), a.end());  
  
for(int i = 1; i <= 5; i++) {  
  
    if(!st.count(i)) {  
        cout << "Missing: " << i;  
        break;  
    }  
}
```



4-208575148360

6. Two Sum

Very important interview question.

Given:

```
2 7 11 15  
target = 9
```

Answer:

```
2 + 7 = 9
```

Hash Map Solution

```
</> C++  
  
vector<int> a = {2,7,11,15};  
  
int target = 9;  
  
unordered_map<int,int> mp;
```

Coding_part_1

```
unordered_map<int,int> mp;

for(int i = 0; i < a.size(); i++) {

    int need = target - a[i];

    if(mp.count(need)) {
        cout << mp[need] << " " << i;
        break;
    }

    mp[a[i]] = i;
}
```

Logic

For every `x`:

```
need = target - x
```

7. First Non-Repeating Character

Input:

aabbcdde

Answer:

c

Code

```
</> C++  
  
string s = "aabbcdde";  
  
map<char,int> freq;  
  
for(char c : s)  
    freq[c]++;  
  
for(char c : s) {
```



```
    for(char c : s) {  
  
        if(freq[c] == 1) {  
            cout << c;  
            break;  
        }  
    }  
}
```

Pattern

1. Frequency count
2. Original string traverse
3. frequency == 1

8. First Repeating Element

Input:

10 5 3 4 3 5

First repeating:

3

`</> C++`

```
vector<int> a = {10,5,3,4,3,5};
```

```
unordered_set<int> st;
```

```
for(int x : a) {
```

```
    if(st.count(x)) {
```

```
        cout << x;
```

```
        break;
```

```
    }
```

```
        cout << x << " ";  
    }  
  
    st.insert(x);  
}
```

9. Common Elements of Two Arrays

```
</> C++  
  
vector<int> a = {1,2,3,4};  
vector<int> b = {3,4,5,6};  
  
unordered_set<int> st(a.begin(), a.end());  
  
for(int x : b) {  
  
    if(st.count(x))  
        cout << x << " ";  
}
```



10. Union of Two Arrays

```
</> C++  
  
vector<int> a = {1,2,3};  
vector<int> b = {3,4,5};  
  
set<int> st;  
  
for(int x : a)  
    st.insert(x);  
  
for(int x : b)  
    st.insert(x);  
  
for(int x : st)  
    cout << x << " ";
```

Output:

```
1 2 3 4 5
```

11. Check Anagram

Two strings:

```
listen  
silent
```

Are anagrams → Yes.

Map Solution

```
</> C++  
  
string a = "listen";  
string b = "silent";  
  
if(a.length() != b.length()) {  
    cout << "Not Anagram";  
    return 0;  
}  
  
map<char,int> freq;  
  
for(char c : a)
```

```
for(char c : a)
    freq[c]++;

for(char c : b)
    freq[c]--;

for(auto p : freq) {

    if(p.second != 0) {
        cout << "Not Anagram";
        return 0;
    }
}

cout << "Anagram";
```

Shortcut

For lowercase English letters:

⌕ C++

```
int freq[26] = {};
```

is even faster.

12. Group Anagrams ★★

Input:

```
eat
tea
tan
ate
nat
bat
```

Groups:

```
eat tea ate  
tan nat  
bat
```

Basic idea:

```
</> C++  
  
map<string, vector<string>> mp;  
  
vector<string> words = {  
    "eat", "tea", "tan", "ate", "nat", "bat"  
};  
  
for(string word : words) {  
  
    string key = word;  
  
    sort(key.begin(), key.end());  
  
    mp[key].push_back(word);  
}
```

```
for(auto group : mp) {  
  
    for(string word : group.second)  
        cout << word << " ";  
  
    cout << endl;  
}
```

Key idea

eat → aet
tea → aet
ate → aet

Same sorted key → same group.

LeetCode 128

13. Longest Consecutive Sequence

Input:

100 4 200 1 3 2

Answer:

4

Because:

1 2 3 4

`unordered_set`

unordered_set

```
</> C++  
  
vector<int> a = {100,4,200,1,3,2};  
  
unordered_set<int> st(a.begin(), a.end());  
  
int longest = 0;  
  
for(int x : st) {  
  
    if(!st.count(x - 1)) {  
  
        int current = x;  
        int length = 1;  
  
        while(st.count(current + 1)) {  
            current++;  
            length++;  
        }  
  
        longest = max(longest, length);  
    }  
}
```

```
cout << longest;
```

Important trick

```
</> C++  
  
if(!st.count(x - 1))
```

means:

Sequence ka starting point mil gaya.

14. Count Subarrays With Given Sum

Input:

```
a = {1,2,3}
k = 3
```

Subarrays:

```
{1,2}
{3}
```

Answer:

2

```
vector<int> a = {1,2,3};

int k = 3;

unordered_map<int,int> mp;

mp[0] = 1;

int prefixSum = 0;
int count = 0;

for(int x : a) {

    prefixSum += x;

    int need = prefixSum - k;

    if(mp.count(need))
        count += mp[need];

    mp[prefixSum]++;
}
```

```
cout << count;
```

Core formula

$$\text{prefixSum} - \text{oldPrefix} = k$$
$$\text{oldPrefix} = \text{prefixSum} - k$$

Isliye:

⌕ C++

```
mp[prefixSum - k]
```

15. Longest Subarray With Sum K

Input:

```
1 2 3 1 1 1 1  
k = 3
```

Longest length:

3

Code

```
</> C++  
  
vector<int> a = {1,2,3,1,1,1,1};  
  
int k = 3;  
  
unordered_map<int,int> mp;  
  
int prefixSum = 0;  
int longest = 0;
```



Coding_part_1

```
int longest = 0;

for(int i = 0; i < a.size(); i++) {

    prefixSum += a[i];

    if(prefixSum == k)
        longest = i + 1;

    if(mp.count(prefixSum - k)) {

        int len = i - mp[prefixSum - k];

        longest = max(longest, len);
    }

    if(!mp.count(prefixSum)) {
        mp[prefixSum] = i;
    }
}

cout << longest;
```